

3D Objects Classification

Marco Minchella, (2131787 / 2159400) and Noemi Stefani, (2095639)[†]

Abstract—We propose a convolutional neural network (CNN) based on the *ORION* model [1]. Several modifications have been made to the network, which has been trained on two different datasets, ModelNet10 and ModelNet40, to evaluate classification and pose estimation accuracy.

In *ORION2*, we compare different activation and optimization functions to assess whether their choice significantly influences the final results.

Subsequently, further structural modifications have been introduced: in *ORION3*, the algorithm's complexity was increased by adding another fully connected layer, while in *ORION4*, skip connections were implemented. Due to unsatisfactory results, *ORION4_2* was finally proposed, synthesizing the best solutions identified during the study.

With this project, we primarily aim to demonstrate that *ORION* is already state of the art and that making modifications, especially by increasing network complexity, does not lead to a significant improvement in accuracy, making this approach not advisable.

Index Terms—3D, Classification, Neural Networks, Convolutional Neural Networks, Voxel grid, Skip Connection.

I. INTRODUCTION

3D object recognition has assumed an important role in several fields of computer vision, with key applications in robotics, augmented reality, and autonomous vehicles. To ensure safe and efficient navigation, perception systems must be able to identify and understand the surrounding three-dimensional environment with high accuracy.

Convolutional neural networks (CNNs) are mainly used in this area; they are a type of feed-forward artificial neural network and exploit a large amount of training data to obtain accurate and generalizable results.

The data used in this paper were processed by voxel grid representation, the latter representing images of objects in three-dimensional grids containing the probability distribution. In this paper, the *ORION* network is implemented, called *ORION1*, then three other CNN architectures are proposed, *ORION2*, *ORION3* and *ORION4* in which each differs in some features from the initial network.

Two datasets are used to train the networks: ModelNet10 and ModelNet40, which provide a wide range of CAD models of real objects, to further compare the accuracy and robustness of the model. Due to limited computing resources for *ORION3*, *ORION4* and *ORION4_2*, we limited ourselves to training it on ModelNet10.

II. RELATED WORK

Traditional 2D object detection methods, based on geometric and statistical techniques such as SURF (Speeded

Up Robust Features) and SVM (Support Vector Machines), have found wide use in classification scenarios. However, these approaches exhibit significant limitations in handling the depth and complex geometry of objects, and are often computationally inefficient. To overcome these challenges, deep learning has been resorted to, particularly with the introduction of convolutional neural networks (CNN), which have proven particularly effective in the capture of complex geometric information [2].

One of the first approaches was 3D ShapeNets, proposed by Wu et al. [3], which introduced the use of Deep Belief Networks (DBNs) to model the geometry of three-dimensional objects through a probabilistic representation on a voxel grid. This work laid the foundation for much subsequent research, including contributing to the creation of ModelNet, one of the first reference datasets for 3D object recognition. However, the DBN approach showed limitations in terms of scalability and computational efficiency, motivating the adoption of more advanced convolutional architectures.

A step forward was made with VoxNet, proposed by Maturana and Scher [4], which applied 3D convolutions on voxel grids to improve accuracy and generalization in three-dimensional object recognition. Although this model represented a major advance over ShapeNets, it was still limited to object classification and was not designed to directly address object detection problems or estimate object pose. In addition, the need to represent data in voxels meant a loss of geometric detail and a high computational cost.

To solve these problems, VoxelNet, proposed by Zhou et al. [5], introduced an end-to-end architecture that combines feature extraction and 3D object detection in a single framework. Unlike VoxNet, VoxelNet uses Voxel Feature Encoding (VFE), a mechanism to aggregate and learn detailed information from the points contained in each voxel before applying 3D convolutions. In addition, VoxelNet integrates a Region Proposal Network (RPN), which enables the generation of 3D bounding boxes, making it particularly suitable for object detection scenarios rather than classification alone.

Further progress in improving rotation invariance was proposed *ORION* by Sedaghat et al. [1], a framework that introduces a loss function term to guide the network to predict not only the class of the object, but also its orientation. This approach has shown that explicit oversight on orientation can improve the robustness of the model, allowing it to learn more generalizable and pose-independent features of objects. Given its remarkable results, we are going to analyze this model with network variations to try to improve the precision and accuracy.

RotationNet was introduced by Kanezaki et al. [6], this

[†]Department of Information Engineering, University of Padova, email: marco.minchella@studenti.unipd.it, noemi.stefani@studenti.unipd.it

method takes advantage of multiple views of an object to improve the estimation of its pose, which has proved to be particularly effective in improving model generalization, especially when training data are limited or have variability in object representation.

Although some approaches have moved towards direct processing of point clouds like PointNet [7], these methods often struggle to capture local geometric relationships efficiently. Hybrid architectures such as PVCNN [8] have attempted to combine the advantages of point clouds and voxel grids. However, voxel-based methods remain dominant due to their structured representation and efficient convolutional operations.

Another significant improvement was achieved with the use of temporal skip connections, initially introduced in the 2D CNNs to mitigate the vanishing gradient problem and then adapted to the 3D CNNs by Köpüklü et al. [9]. This approach has been shown to improve the training efficiency and stability of deeper networks, reducing computational cost without compromising the quality of object representation.

Finally, the evolution of 3D datasets has had a fundamental impact in the development of the latest models. In addition to ModelNet, datasets such as ShapeNet and ScanNet have become gold standards for performance evaluation of deep learning networks, offering real and synthetic objects with detailed annotations for classification and segmentation tasks. In our work, we focus on a voxel grid representation, exploring new architectural strategies to improve the quality of the spatial representation and optimize the accuracy of the model in recognizing and estimating the orientation of 3D objects.

III. PROCESSING PIPELINE

What we did was try different hyperparameters of the neural networks with both MN10 and MN40 datasets, after that we proceeded to try to modify the neural network to see if small architectural tweaks would improve in any way the classification performances.

We ran the architecture four different times for each different Model Net dataset everytime with a different combination of activation function and optimizers, namely ELU, Tanh and Adam, RMSprop.

The reason for these choices lands on the hope to see noticeable differences in performance between runs especially when swapping activation functions since Tanh has two big flat intervals that may lead us to a higher chance of having a vanishing gradient issues.

After different runs and after figuring out the best optimizer and activation function, we attempted to improve the architecture via two different solutions:

- 1) Adding an extra fully connected layer
- 2) Adding skip connections in the convolutional layers

In particular when dealing with skip connections we decided to explore two possibilities, namely a skip connection every convolutional layer and a single skip connection that from the input to the last convolutional layer.

These different solutions will be better explored in the "Architectures" section.

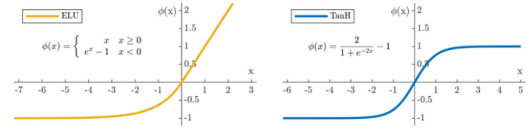


Fig. 1: Plots of the activation functions employed.

IV. SIGNAL AND FEATURES

The ModelNet10 and ModelNet40 datasets we used for this project were provided by the Princeton University website and are composed of synthetic CAD models.

In order to feed the data into the neural network we had to perform some pre-processing steps with the goal of converting both datasets into voxel grids and to do that we made use of the MATLAB code provided by the Albert-Ludwigs-Universitat Freiburg GitHub repository [10] that not only performed format conversions but also performed train/test splits in all the folders of the various objects.

Datasets original train/test split		
Dataset	Train samples	Test samples
ModelNet 10	3991	908
ModelNet 40	9843	2468

We managed to modify this split by slightly changing the code and then performing a rough 60/20/20 train/validation/test division by manually moving the output of the code into 3 different folders for each object class.

After performing the dataset loading we transformed it into 36x36x36 voxel grids (Figure 2) which we then cropped to have 32x32x32 sizes.

Every object in the dataset is made up of several point of views, to be precise 12 different rotations around the z axis (30 degrees each) and is manually labeled with the correct class and orientation [1]. It has to be said that one could have enhanced the number of rotations in order to have a richer training set, but we felt like 12 points of view per object were already a good choice to classify correctly most of the objects.

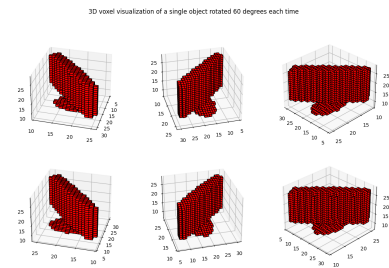


Fig. 2: A single object in the ModelNet 40 dataset rotated 60 degrees per subplot.

V. LEARNING FRAMEWORK

We now proceed to analyze the four different classes, each of them with a certain architecture modification in order to study how accuracies and precisions change in every case.

ORION1 (Figure 3) is the class which was implemented exactly like in the reference paper in order to use it as reference for change of activation functions, optimizers or structure.

This model presents a 3D convolutional neural network for feature extraction followed by a fully connected section which produces two independent outputs. The convolutional network is made of four layers, each of them with $3 \times 3 \times 3$ kernel size and increasing values of 3D dropout (from 0.2 to 0.6) to prevent overfitting, every convolutional layer is followed by a batch normalization layer to stabilize the activations and improve convergence. The last convolutional layer includes a Max-Pooling operation which reduces the spacial dimensionality allowing the model to focus on the most relevant aspect of the input. After feature extraction the result of the convolutional layers is flattened and passed to an hidden fully connected layer with 128 neurons which serves as latent representation that the model uses to produce its two outputs: the object class prediction and the object pose estimation.

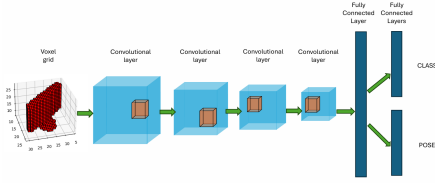


Fig. 3: Scheme of the original neural network's architecture.

The loss is calculated by averaging the multinomial cross entropies losses of the class and pose outputs

$$\mathcal{L} = \frac{\mathcal{L}_c + \mathcal{L}_o}{2}$$

where $\mathcal{L}_c$ is the class loss while $\mathcal{L}_o$ is the orientation loss.

It's important to note that orientation score is crucial when it comes to test phase since we feed multiple rotations of the test object to the network and get a final consensus on the class label based on the votes we obtain from each inference pass

$$c_{final} = \arg \max_k \sum_r S_k(x_r)$$

where S_k is the score the network assigns to the object at its k -th node of the object category output layer and x_r is the test input with the rotation index r [1].

To guarantee a more stable training phase the CNN weights were initialized with the Kaiming Normal technique, especially effective with networks with ReLU activations since it helps keep the gradients' variance stable during propagation. The weights of the fully connected part are initialized with a

normal distribution with standard deviation of 0.01, while the biases are set to zero in all the layers.

ORION2 differs slightly from the previous architecture since it employs different activation functions, namely ELU and Tanh and was used to perform the different runs on MN10 and MN40 with the different optimizers (Adam and RMSProp).

ORION3 was the first attempt at improving the architecture and the solution we choose was making the fully connected section deeper by adding a single hidden layer with 256 neurons between the last convolutional layer and the original 128 neurons layer with both of the hidden layers have a dropout of 0.4 and have no batchnorm.

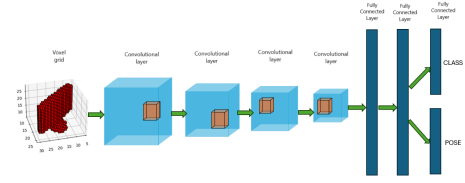


Fig. 4: Scheme of the ORION 3 architecture.

The **ORION4** architecture employs skip connections in order to allow a better flow of the gradient during backpropagation. We decided to build two different version of this network positioning residual blocks in different places: in the first (**ORION4**, figure 5) we used a skip connection for every convolutional block, while in the second (**ORION4_2**) we used a single skip connection from the input directly to the output of the last convolutional layer.

Each convolutional block is made up by 2 routes:

- Main path, which, as stated before, applies 3D convolutions with $3 \times 3 \times 3$ kernels, batch norm, activation function and dropout fixed at 0.2.
- Skip connection, which allows us to preserve the original information and allows the gradient to flow without going through a lot of activations

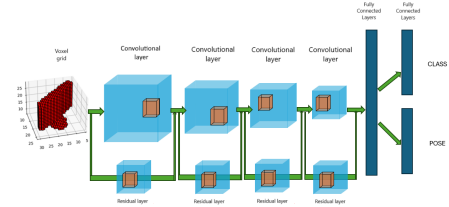


Fig. 5: Scheme of the first version of the ORION 4 architecture.

The results obtained while testing the proposed model were very poor, with low accuracies and high losses even when tweaking parameters such as dropout (more on this in the VI section).

For the last architecture **ORION4_2** (6) not only we decided

to use a single skip connection, but we also opted to employ the two hidden layer architecture from **ORION3**, basically trying to take the "best from both worlds". While building this network we faced tensor mismatching problems when adding the main output and the shortcut output, so we decided to solve it by padding the output of the shortcut to allow the summation.

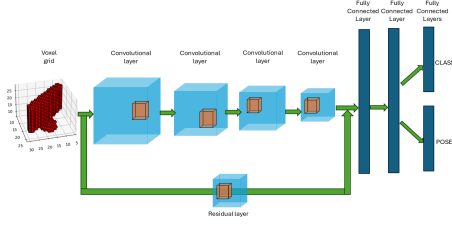


Fig. 6: Scheme of the second version of the ORION 4 architecture.

VI. RESULTS

In this section we provide the results of the different runs we performed on the different datasets and architectures. We logged our results in Wandb.

A. Model Net 10

We start by showing the results of the runs on the 4 different combinations of activation functions and optimizers on the **ORION2** neural network. One can clearly see how training

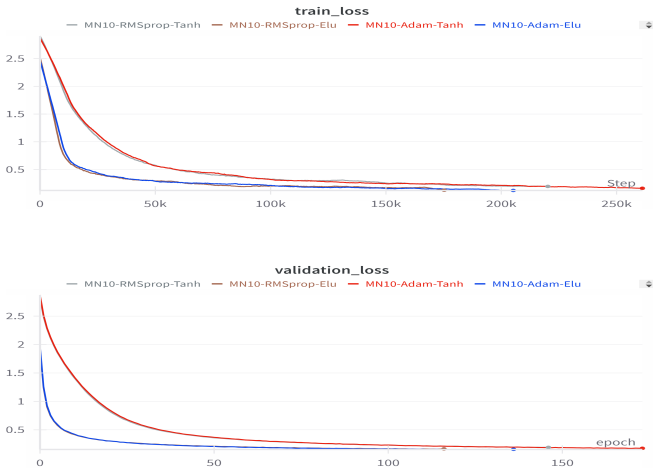


Fig. 7: Train and validation losses plots comparison for MN10

and validation losses have similar trends for all cases, however we notice a big difference between the models that employ an ELU activation function with respect to models that use a hyperbolic tangent: while in the long run the losses are similar in values, when using ELU we get a much faster convergence towards minimum loss values.

We now focus our attention on the accuracies in figure 8 and

note how the plots show trends that are coherent to the ones seen in figure 12 with the ELU models performing slightly better and reaching high values quicker.

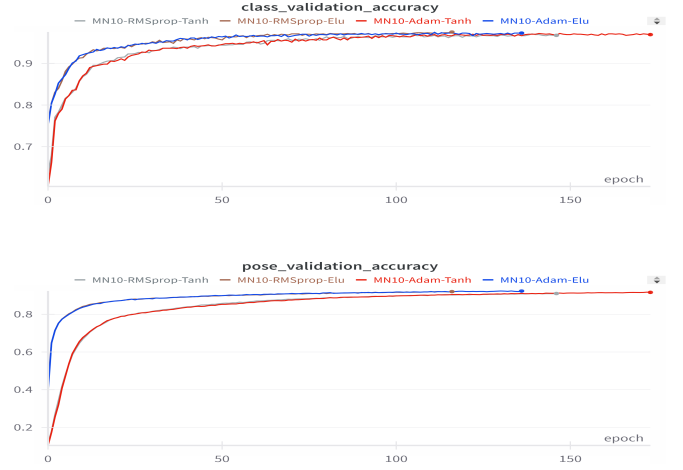


Fig. 8: Validation accuracies for the MN10 dataset using ORION2

Test accuracy values with the MN10 dataset			
Activation	Optimizer	Class	Pose
ELU	Adam	92.8 %	85.13 %
Tanh	Adam	91.57 %	84.48 %
ELU	RMSProp	93.15 %	84.99 %
Tanh	RMSProp	92.45 %	84.29 %

B. Model Net 40

We move on to the Model Net 40 dataset, a much richer object set which presents 40 different classes instead of 10. Our goal here was to check how the neural network performed when presented with a more challenging task and see if it would yield us satisfactory results as it did when testing on MN10. For this dataset we followed the same procedures and steps as before, training the same four different combinations of activations and optimizers.

It's immediate to see how the model finds it more difficult to minimize the total loss with this richer dataset since it seems to plateau on higher values if compared with the MN10 losses. At the same time we can notice similar behaviours repeated, in the sense that once again the models employing ELU activation functions converge to similar values quicker and in this case seem to perform better during training time. Let us check if this is true when running inferences.

Test accuracy values with the MN40 dataset			
Activation	Optimizer	Class	Pose
ELU	Adam	86.91 %	83.89 %
Tanh	Adam	86.3 %	82.59 %
ELU	RMSProp	87.11 %	83.64 %
Tanh	RMSProp	85.93 %	83.04 %

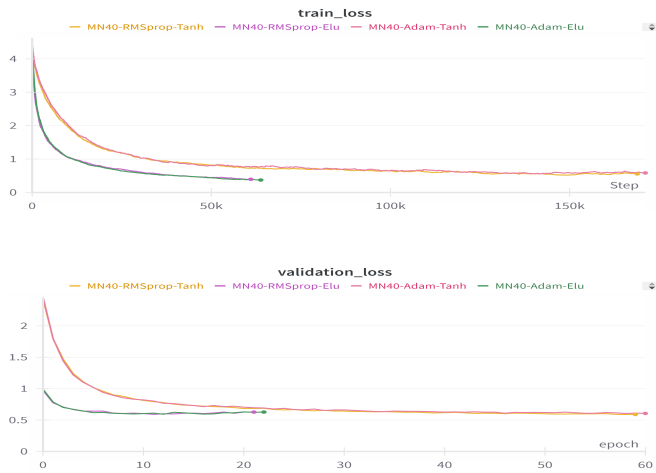


Fig. 9: Train and validation losses plots comparison for MN40

As expected the network struggles to keep the same level of accuracies with this more complex task, however it's interesting to see how the pose accuracies have suffered less loss of accuracy if compared to the object classes classification performances, this is probably due to the fact that we are still dealing with the same number of orientations as before while we have quadrupled the number classes. After these 8 runs between the 2 different datasets and after looking at the various train, validation and test results we decided to use the combination of ELU activation function and Adam optimizer. The choice of the activation function was made by picking the one that clearly gave us better performances and also had less chances of having a vanishing gradient problems, while the choice of the optimizer was made knowing that Adam is one of the most popular choices when it comes to training CNNs and in neural networks general [11]. Due to our limited computing resources at this point we opted to test the following modified architectures only on the smaller MN10.

C. ORION3

As stated in section V the ORION3 network was our first attempt at modifying the original architecture by adding a new hidden fully connected layer with 256 neurons, batch norm and a dropout of 0.4. We expected to have results which improved slightly on the original network but at training time the loss values converge at roughly to the same values and do so a bit more slowly. The same can be said for the validation accuracies: in the ORION2 architecture with ELU and Adam the maximum values were around 97 % for class and 92 % for pose, while in ORION3 we get 97 % for class and 90 % for pose.

Test accuracy comparison		
Architecture	Class	Pose
ORION2	92.8 %	85.13 %
ORION3	92.8 %	85 %

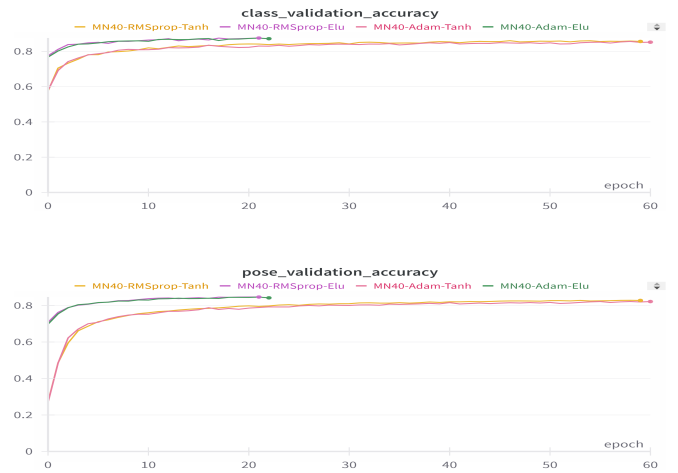


Fig. 10: Validation accuracies for the MN10 dataset using ORION2

During inference we still found no improvement on the classification task, so given the almost identical results between architectures we opted to skip the graph plots since they offer little to no meaningful additional insight. This first attempt made the network more complex without giving any kind of sensible improvement or change so we moved on to ORION4.

D. ORION4

The next step for modifying the architecture was relying on residual blocks in order to improve performances and gradient flow. Our idea was to put the skip connections in the convolutional part, however we decided to look at two opposite extreme solutions, namely a skip connections every convolutional layer and a skip connection starting from the input and ending before the fully connected section (figures 5 and 6)

1) *Multiple skip connections*: This first version did not work as intended and was stopped only after a few epochs due to the fact that the loss stayed stationary at high values and the validation precisions were, as one could expect, extremely low sitting at around 27 % for class accuracy and 2 % for pose accuracy. We immediately concluded that this attempt was a complete failure since it was extremely more complex and was never able to have any kind of improvement even from its first epoch.

2) *Single skip connection and double hidden FC layer*: After a first run attempt where we kept the skip connection and a single hidden layer we discovered that performing a skip connection from input to the output of the last convolutional block on the architecture of ORION3 yielded surprisingly good results with respect to the original structure, at times even marginally better. The training time was cut short due to the fact that it never reached early stopping conditions and we couldn't progress after 24 hours of elaboration with the resources at hand.

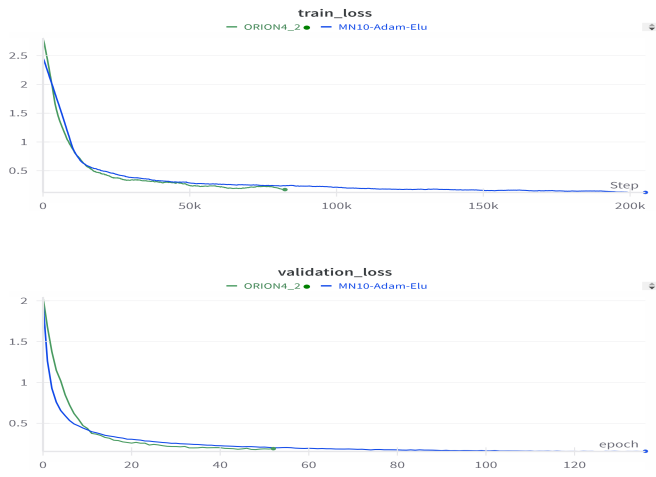


Fig. 11: Train and validation losses comparison between ORION4_2 and ORION2. After a certain number of epochs ORION4_2 presents slightly better results.

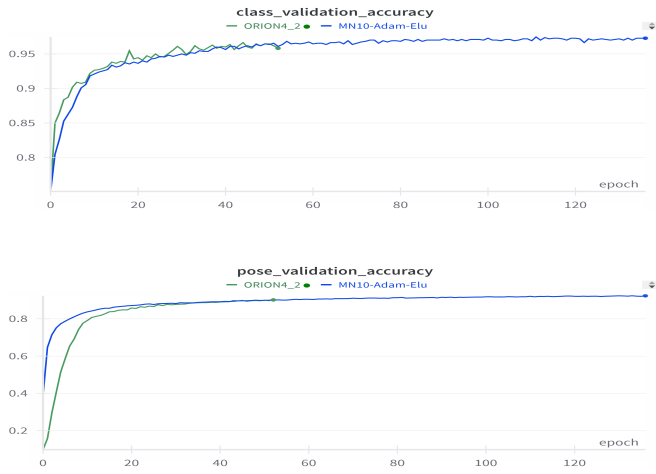


Fig. 12: Validation accuracies comparison between ORION4 and ORION2.

During validation the accuracies were quite similar with no clear winner between the architectures, this is also reflected if we compare test accuracies

Test accuracy comparison		
Architecture	Class	Pose
ORION2	92.8 %	85.13 %
ORION4_2	92.98 %	84.32 %

This last architecture was the most promising one and it would be interesting to see its performances with a longer training time, however as stated before this was not possible with the computing resources at hand.

VII. CONCLUDING REMARKS

In this project, we tackled the problem of 3D object recognition by implementing the **ORION** architecture and com-

paring four different variants: **ORION2**, **ORION3**, **ORION4**, and **ORION4_2**. The analysis highlighted the importance of choosing the activation function, showing that ELU offers superior performance compared to Tanh, while the choice of optimizer had a less significant impact on the results.

The introduction of a double layer increased the complexity of the model without bringing significant improvements. The skip connections, implemented to improve gradient flow, did not yield the expected results as reported in [9], probably due to differences in their application. In the reference paper, temporal connections are used in networks that process sequential information over time, whereas, in our case, they were applied statically to spatial features. This distinction could explain the difference in the results obtained.

However, with ORION4_2, where only one skip connection was added and the number of layers was increased, we achieved good results, particularly regarding the accuracy of object pose estimation, at the cost of increased model complexity.

Future developments could further explore these architectural aspects, investigating the use of point cloud-based representations in addition to voxel grids to further improve model performance and generalization.

REFERENCES

- [1] N. Sedaghat, M. Zolfaghari, E. Amiri, and T. Brox, "Orientation-Boosted Voxel Nets for 3D Object Recognition," in *Proceedings of the British Machine Vision Conference (BMVC)*, 2017.
- [2] A. Pravalika, M. F. Hashmi, and A. Gupta, "Deep Learning Frontiers in 3D Object Detection: A Comprehensive Review for Autonomous Driving," *IEEE Access*, vol. 12, pp. 173936–173980, 2024.
- [3] Z. Wu, S. Song, A. Khosla, F. Yu, L. Zhang, X. Tang, and J. Xiao, "3D ShapeNets: A Deep Representation for Volumetric Shapes," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1912–1920, 2015.
- [4] D. Maturana and S. Scherer, "VoxNet: A 3D Convolutional Neural Network for Real-Time Object Recognition," in *Proceedings of the IEEE/RISJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 922–928, 2015.
- [5] Y. Zhou and O. Tuzel, "VoxelNet: End-to-End Learning for Point Cloud Based 3D Object Detection," in *Proceedings of the 31st Conference on Neural Information Processing Systems (NeurIPS)*, pp. 4490–4499, 2017.
- [6] A. Kanezaki, Y. Matsushita, and Y. Nishida, "RotationNet: Joint Object Categorization and Pose Estimation Using Multiviews from Unsupervised Viewpoints," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 5010–5019, 2018.
- [7] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 652–660, 2017.
- [8] Z. Liu, H. Tang, Y. Lin, and S. Han, "Point-Voxel CNN for Efficient 3D Deep Learning," in *Proceedings of the 33rd Conference on Neural Information Processing Systems (NeurIPS)*, 2019.
- [9] O. Köpüklü, A. Zheng, H. Xu, and G. Rigoll, "Temporal Skip Connections for Dissected 3D CNNs," in *Proceedings of the 33rd Conference on Neural Information Processing Systems (NeurIPS)*, 2019.
- [10] N. Sedaghat, M. Zolfaghari, E. Amiri, and T. Brox, "Orientation-boosted voxel nets for 3d object recognition," in *British Machine Vision Conference (BMVC)*, 2017.
- [11] A. Gupta, "MS Windows NT kernel description." <https://www.analyticsvidhya.com/blog/2021/10/a-comprehensive-guide-on-deep-learning-optimizers>.